Chapter 02

2장

느려진 서비스, 어디부터 봐야 할까

이 장에서 다룰 내용

- 처리량과 응답 시간
- 병목 지점과 확장
- DB 커넥션 풀
- 서버 캐시
- 정적 자원과 캐시/CDN
- 대기 처리

2 처리량과 응답 시간

앱을 실행했을 때 로딩 중인 이미지가 오래 표시되거나 통신사 사이트에서 요금 정보를 조회하는 데 10초 이상 걸린다면 흔히 성능이 나쁘다고 말한다. 성능이 저하되면 가장 눈에 띄는 현상은 결과가 늦게 표시되는 것이다. 너무 오래 걸려서 타임아웃(시간 초과) 에러가 발생하기도 한다.

사용자는 무언가를 실행할 때 동작하기까지 걸린 시간으로 성능을 판단하지만 실제로는 다양한 지표가 성능과 관련되어 있다. 네트워크 속도, 디스크 속도, 메모리 크기, 디바이스(스마트폰)의 CPU 속도 등이 여기에 해당한다. 이런 다양한 지표 중에서 서버 성능과 관련 있는 중요한 지표를 2가지 꼽자면 응답 시간과 처리량을 들 수 있다.

응답 시간

먼저 응답 시간[1]을 알아보자. 응답 시간은 사용자의 요청을 처리하는 데 걸리는 시간을 의미한다. [그림 2.1]은 전형적인 API 호출 과정을 나타낸 것이다. 이 그림에서 API를 호출(과정 1)하고 전체 JSON 응답을 받을 때까지(과정 1.3) 소요된 시간이 바로 응답 시간이다.

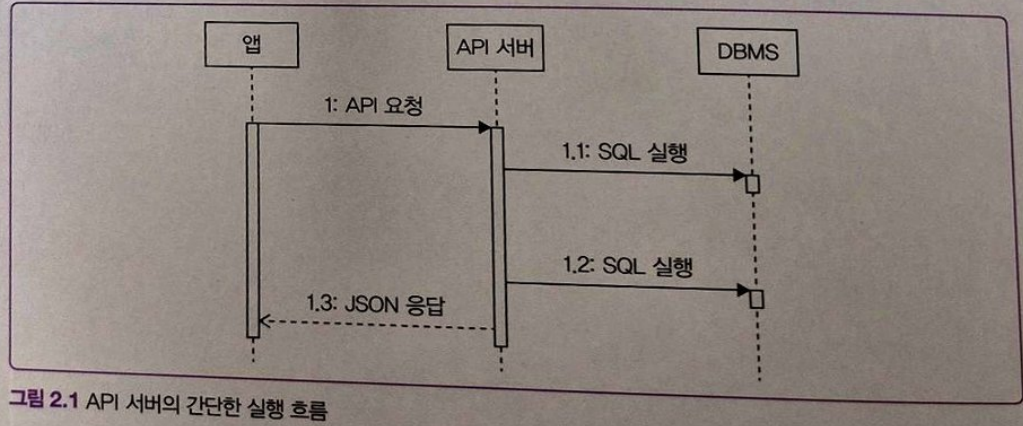


그림 2.1 API 서버의 간단한 실행 흐름

1 성능 측정 도구에서 응답 시간은 주로 response time으로 표시한다.

응답 시간을 구성하는 요소에 대해 좀 더 자세히 살펴보자. 하나의 API 요청을 처리하는 데 걸리는 전체 시간은 [그림 2.2]와 같이 구성된다.

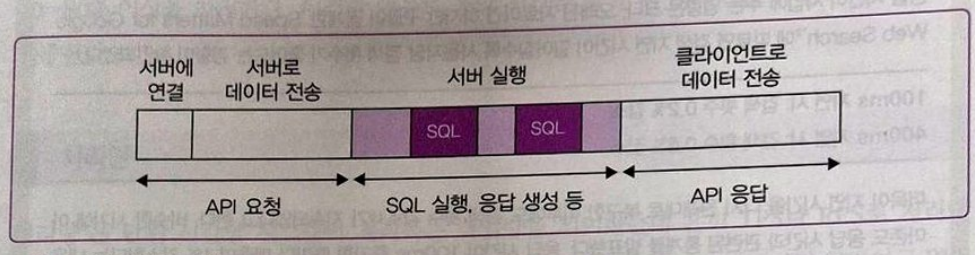


그림 2.2 응답 시간의 구성

클라이언트(앱이나 브라우저)가 서버로 요청을 보내는 과정은 크게 2단계로 이루어진다.

1. 서버에 연결: TCP를 이용해서 서버에 연결한다.
2. 데이터 전송: 정해진 규칙(프로토콜)에 따라 데이터를 서버에 전송한다. 예를 들어, HTTP 프로토콜에 따라 POST 방식으로 JSON 데이터를 보낼 수 있다.

서버는 로직을 실행한 다음에 응답 데이터를 클라이언트에 전송한다. 응답 데이터를 전송할 때는 API 요청 과정에서 서버와 연결된 소켓을 이용한다.

응답 시간은 다음과 같이 2가지로 나누어 측정하기도 한다.

- TTFB(Time to First Byte): 응답 데이터 중 첫 번째 바이트가 도착할 때까지 걸린 시간
- TTLB(Time to Last Byte): 응답 데이터의 마지막 바이트가 도착할 때까지 걸린 시간

응답 데이터의 크기가 작다면 TTFB와 TTLB의 차이가 크지 않다. 하지만 파일 다운로드처럼 전송할 데이터가 크거나 네트워크 속도가 느리면 TTFB와 TTLB의 차이가 커질 수 있다. 따라서 서버 성능을 올바르게 평가하려면 데이터 특성이나 네트워크 환경을 고려해 TTFB와 TTLB 중 적절한 지표를 선택해 측정해야 한다.

응답 시간은 0.025초처럼 1초보다 짧을 때가 많다. 그래서 성능 측정을 위해 응답 시간을 잴 때는 1/1000초인 밀리초 단위를 사용한다. 밀리초를 표현할 때는 10ms처럼 'ms'를 뒤에 붙인다.

Column

응답 시간은 중요하다

응답 시간이 사업에 주는 영향은 크다. 오래된 자료이긴 하지만 구글이 공개한 'Speed Matters for Google Web Search'[2]에 따르면 검색 지연 시간이 길어질수록 사용자당 검색 횟수가 줄어드는 경향이 확인되었다.

100ms 지연 시: 검색 횟수 0.2% 감소
400ms 지연 시: 검색 횟수 0.6% 감소

더욱이 지연 시간을 다시 원래대로 복구한 이후에도 검색 횟수 감소세가 지속되었다고 한다. 비슷한 시기에 아마존도 응답 시간과 관련된 통계를 발표했다. 응답 시간이 100ms 증가할 때마다 매출이 1% 감소한다는 내용이었다.

응답 시간이 증가하면 트래픽과 매출이 줄어든다. 반대로 응답 시간을 줄여야 트래픽과 매출이 줄지 않는다. 응답 시간을 줄인다고 해서 매출이 반드시 증가하는 것은 아니지만 응답 시간이 길어질수록 서비스에 부정적인 영향을 주는 것은 확실하다.

[그림 2.2]에서 봤듯이 응답 시간은 API 요청 전송 시간, 서버의 처리 시간, API 응답 전송 시간으로 나뉜다. 서버 개발자는 주로 서버의 처리 시간을 확인한다. 서버 처리 시간은 다음과 같은 요소를 포함한다.

- 로직 수행(if, for 등)
- DB 연동(SQL 실행)
- 외부 API 연동
- 응답 데이터 생성(전송)

이 중에서도 DB 연동과 외부 API 연동이 큰 비중을 차지한다. 다음은 필자가 경험한 서비스에서 실제 한 요청의 처리 시간을 측정한 결과다.

- 전체 처리 시간: 348ms
- API 연동 1(외부 네트워크에 존재하는 API 호출): 186ms(53%)
- API 연동 2(내부 네트워크에 존재하는 API 호출): 44ms(13%)
- DB 연동(SQL 실행 6회): 101ms(29%)
- 로직 수행: 17ms(5%)

2 https://blog.research.google/2009/06/speed-matters.html

API 연동 2번만으로 전체 처리 시간의 66%나 차지한다. DB 연동도 29%를 차지한다. 반면 로직 수행 시간은 5%에 불과하다. API 연동이 없는 경우라면 DB 연동이 전체 실행 시간의 70~80% 이상을 차지한다. 이러한 이유로 응답 시간을 줄일 때 DB 연동과 API 연동 시간에 집중한다.

처리량

처리량[3]은 단위 시간당 시스템이 처리하는 작업량을 의미하는데 흔히 TPS나 RPS로 처리량을 나타낸다. TPS는 초당 트랜잭션 수를 의미하는 transaction per second의 약자이고 RPS는 초당 요청 수를 의미하는 request per second의 약자이다. 서버의 성능 지표를 표현할 때 TPS나 RPS를 구분 없이 혼용하기도 하지만 이 책에서는 TPS를 사용한다.

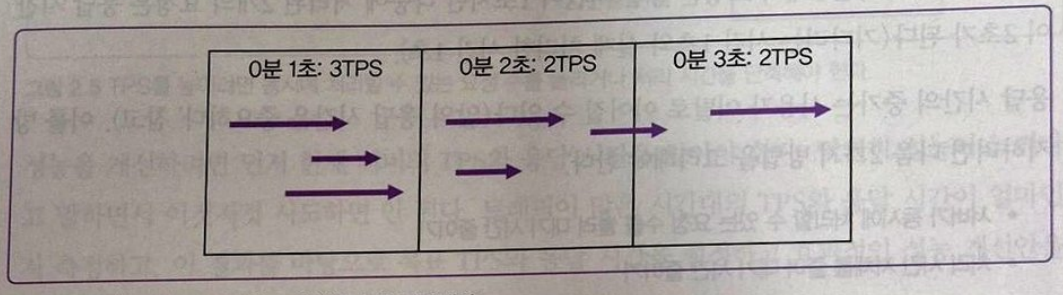


그림 2.3 TPS는 초당 처리한 트랜잭션 수를 나타낸다.

[그림 2.3]에서 화살표는 요청을 의미하고 화살표의 길이는 응답 시간을 나타낸다. 0분 1초 구간에는 완료된 요청이 3개이므로 이 구간의 TPS는 3이다. 0분 2초 구간에서 완료된 요청은 2개이므로 TPS는 2가 된다.

최대 TPS는 시스템이 처리할 수 있는 최대 요청 수를 의미한다. 예를 들어, 서버가 한 번에 5개의 요청을 처리할 수 있다고 가정하자. 이때 요청당 처리 시간이 1초라면 최대 TPS는 5가 된다. 동시에 들어오는 요청 수가 최대 TPS를 초과하면 서버는 초과한 요청을 나중에 처리한다.

---

3 성능 측정 도구에서 처리량은 throughput으로 표현한다.

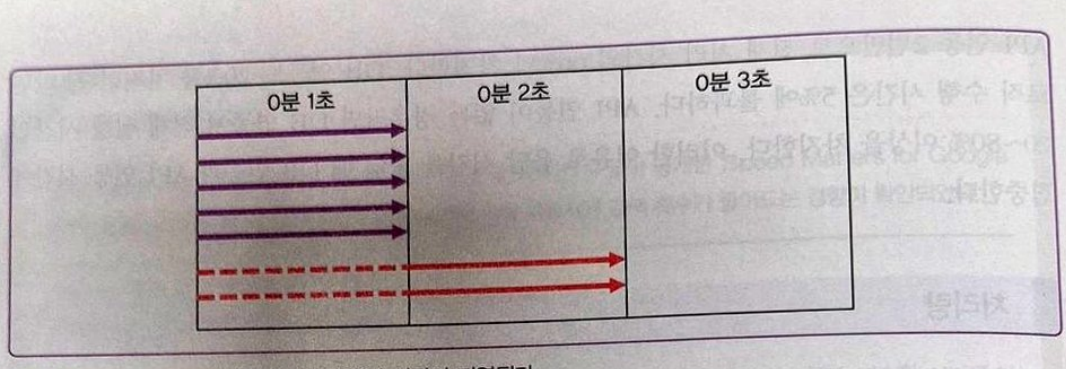


그림 2.4 요청이 최대 TPS를 초과하면 응답 시간이 지연된다.

[그림 2.4]는 최대 TPS를 초과하는 요청이 들어왔을 때 사용자 입장에서 응답 시간이 증가하는 문제를 보여준다. 최대 TPS가 5인 서버에 동시에 7개의 요청이 들어오면 이 중 5개만 바로 처리할 수 있다. 나머지 2개는 먼저 실행된 5개의 요청이 끝난 후에야 처리할 수 있다. 사용자 입장에서 먼저 처리된 5개의 요청은 응답 시간이 1초지만 나중에 처리된 2개의 요청은 응답 시간이 2초가 된다(기다리는 시간 1초와 실제 처리한 시간 1초).

응답 시간의 증가는 사용자 이탈로 이어질 수 있다(앞의 '응답 시간은 중요하다' 참고). 이를 방지하려면 다음 2가지 방법을 고려해야 한다.

- 서버가 동시에 처리할 수 있는 요청 수를 늘려 대기 시간 줄이기
- 처리 시간 자체를 줄여 대기 시간 줄이기

두 방법을 적용하면 TPS를 높일 수 있다.

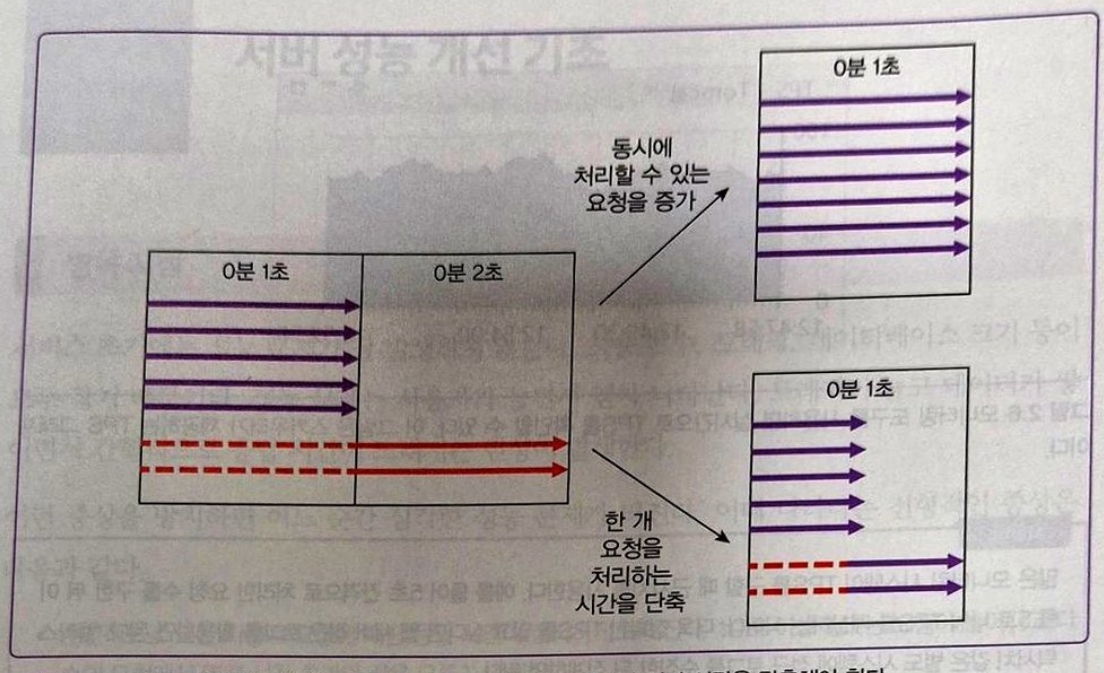


그림 2.5 TPS를 높이려면 동시에 처리할 수 있는 요청 수를 늘리거나 처리 시간을 단축해야 한다.

성능을 개선하려면 먼저 현재 서버의 TPS와 응답 시간을 알아야 한다. 막연히 성능이 느리다고 말하면서 이것저것 시도하면 안 된다. 트래픽이 많은 시간대의 TPS와 응답 시간이 얼마인지 측정하고, 이 결과를 바탕으로 목표 TPS와 응답 시간을 설정하고 효과적인 성능 개선안을 도출해야 한다.

TPS를 확인하는 가장 간단한 방법은 모니터링 시스템을 활용하는 것이다. 스카우터, 핀포인트, 뉴렐릭 같은 도구를 사용하면 실시간 TPS뿐 아니라 과거 특정 시점의 TPS도 확인할 수 있다.

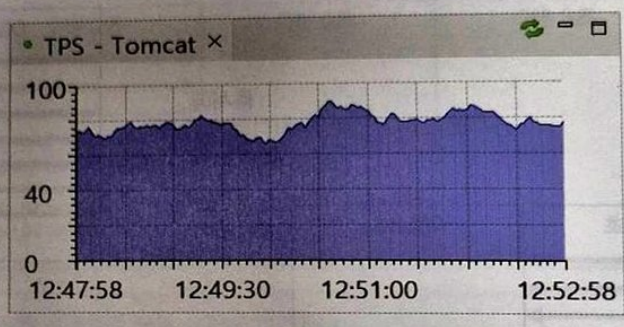


그림 2.6 모니터링 도구를 사용하면 실시간으로 TPS를 확인할 수 있다. 이 그림은 스카우터가 제공하는 TPS 그래프이다.

Memo

많은 모니터링 시스템이 TPS를 구할 때 근사치를 사용한다. 예를 들어 5초 간격으로 처리한 요청 수를 구한 뒤 이를 5로 나눠 TPS를 계산하는 식이다. 더욱 정확한 TPS를 알고 싶다면 웹 서버 접근 로그를 활용하면 된다. 엘라스틱서치 같은 별도 시스템에 접근 로그를 수집한 뒤 집계하면 된다.

엘라스틱서치나 핀포인트 같은 도구가 없다면 접근 로그를 파싱해서 TPS를 구해도 된다. 다소 번거롭긴 하지만 약간의 리눅스 명령어와 코딩만으로도 TPS를 산출할 수 있다.

03 서버 성능 개선 기초

병목 지점

서비스 초기에는 성능 문제가 잘 발생하지 않는다. 사용자 수, 트래픽, 데이터베이스 크기 등이 모두 작기 때문이다. 성능 문제는 사용자가 늘면서 점차 나타난다. 트래픽이 늘고 데이터가 쌓이면서 간헐적으로 응답 시간이 느려지는 현상이 발생한다.

이런 증상을 방치하면 어느 순간 심각한 성능 문제가 터진다. 이때 나타나는 전형적인 증상은 다음과 같다.

- 순간적으로 모든 사용자 요청에 대한 응답 시간이 심각하게 느려진다. 10초 이상 걸리는 요청이 늘어나고 다수의 요청에서 연결 시간 초과와 같은 오류가 발생한다.
- 서버를 재시작하면 잠시 괜찮다가 다시 응답 시간이 느려지는 현상이 반복된다.
- 트래픽이 줄어들 때까지 심각한 상황이 계속된다.

트래픽이 증가하면서 성능 문제가 발생하는 주된 이유는 시스템이 수용할 수 있는 최대 TPS를 초과하는 트래픽이 유입되기 때문이다. 시스템이 제공할 수 있는 최대 TPS를 높이지 않으면 증가하는 트래픽을 적절히 처리할 수 없다.

TPS를 높이려면 먼저 성능 문제가 발생하는 지점을 찾아야 한다. 문제 지점을 찾는 간단한 방법은 처리 시간이 오래 걸리는 작업을 식별하는 것이다. 성능 문제는 응답 시간이 길어지면서 발생하는 경우가 많기 때문이다. 경험에 의해 감으로 오래 걸릴 것 같은 코드를 추측할 수도 있지만 가능하다면 실제 실행 시간을 측정해야 한다. 이때 모니터링 도구가 유용하다. 대부분의 모니터링 도구는 실행 시간 추적 기능을 제공하므로 이 기능을 활용하면 성능 문제가 발생하는 시점에 어떤 코드에서 실행 시간이 오래 걸렸는지 찾을 수 있다. 적절한 모니터링 도구가 없다면 로그라도 남겨야 한다. 의심되는 코드의 실행 시간을 로그로 남겨두면 나중에 성능 문제가 다시 발생했을 때 개선할 부분을 찾는 데 도움이 된다.

일반적인 서버는 DB나 다른 API와 연동한다. 필자의 경험상 성능 문제는 주로 DB나 외부 API를 연동하는 과정에서 발생했다.

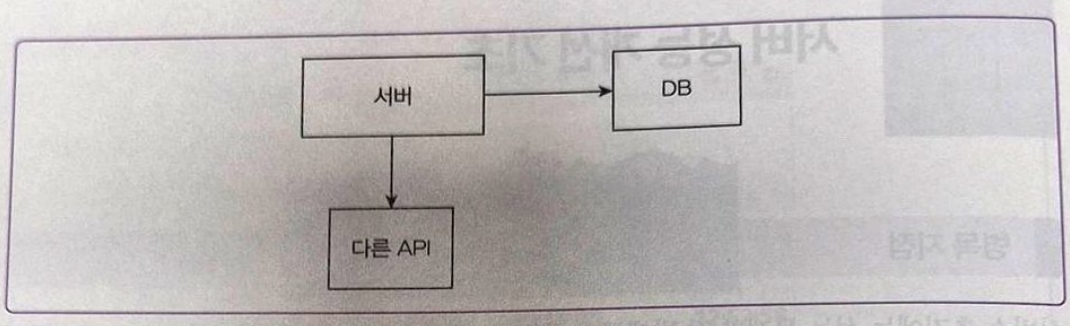


그림 2.7 서버 성능 문제는 주로 DB 연동이나 API 연동 과정에서 발생한다.

여기에서는 서버 관점에서 처리량과 응답 시간을 개선하는 방법을 중심으로 살펴보자. DB와 관련해 서버 개발자가 알아야 할 기초 내용은 3장에서, 외부 연동과 관련된 내용은 4장에서 다룰 것이다.

수직 확장과 수평 확장

성능 문제를 일으키는 원인을 찾았다면 빠르게 적용할 수 있는 개선안을 도출해야 한다. 사용자가 서비스를 이용하지 못하는 상황에서 이를 방치한 채 시간만 오래 걸리는 개선 방안을 시도할 수는 없다. 일단 급한 불을 끄고 나서 근본적인 해결책을 모색해야 한다.

급한 불을 끄는 방법 중 하나는 수직 확장scale-up을 하는 것이다. 수직 확장은 CPU, 메모리, 디스크 등의 자원을 증가시키는 것을 말한다. 더 빠른 CPU로 바꾸거나 CPU 코어수를 늘리고 메모리를 확장하고 디스크를 SSD로 바꾸는 것만으로도 성능이 개선될 수 있다. 클라우드 환경에서 비교적 빠르게 시도할 수 있는 방법이다.

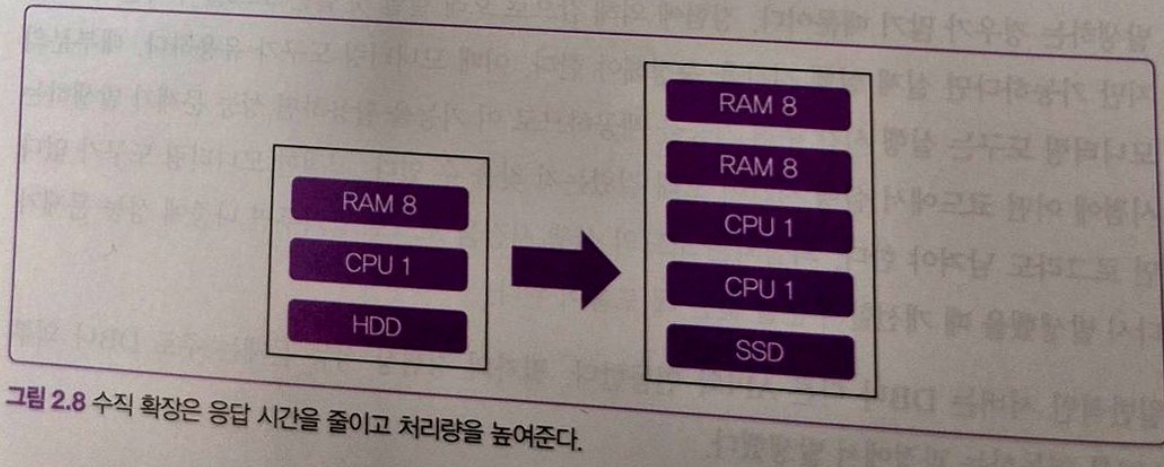


그림 2.8 수직 확장은 응답 시간을 줄이고 처리량을 높여준다.

필자도 수직 확장을 통해 긴급한 상황을 해결한 경험이 있다. 예전에 M사 서비스에 장애가 발생해서 급히 대응한 적이 있다. M사 서비스는 사용자가 조금만 몰려도 전체 시스템이 먹통이 되는 문제가 있었다. 원인은 DB 성능에 있었다. 실행되는 쿼리, 테이블 설계 등 여러 면에서 문제가 있었으며 근본적으로 해결하려면 많은 시간이 필요했다. 그래서 우선 진행한 작업이 DB 장비의 수직 확장이었다. 고성능 CPU로 교체하고 메모리도 증설했다. 이를 통해 일단 서비스를 재개할 수 있었다. 완전히 중단되는 것보다 다소 느리더라도 서비스를 제공하는 게 낫기 때문이다. 또한 문제 원인을 해소할 방안을 찾고 적용할 수 있는 시간도 벌 수 있었다.

- 수직 확장이 DB에서만 효과가 있는 것은 아니다. 서버도 메모리 부족으로 성능 문제가 발생할 수 있다. 예를 들어 서버 메모리가 512MB일 때 동시에 처리할 수 있는 요청 개수가 200개라면 동시 요청이 250개로 늘어나는 것만으로도 응답 시간이 증가하기 시작한다. 이때 서버 메모리를 1GB로 증설하기만 해도 TPS를 늘릴 수 있다.

수직 확장은 즉각적인 효과를 바로 얻을 수 있지만 트래픽이 지속해서 증가하면 언젠가 결국 또다시 성능 문제가 발생한다. 그때마다 수직 확장을 반복할 수는 없다. 수직 확장은 비용이 많이 들기 때문이다. 또한 한 대의 장비가 감당할 수 있는 용량에도 한계가 있다. 따라서 트래픽이 증가하면 서버를 추가로 투입해 TPS를 높이는 방법도 고려해야 한다. 이렇게 서버를 늘리는 방법을 수평 확장scale-out이라고 한다.

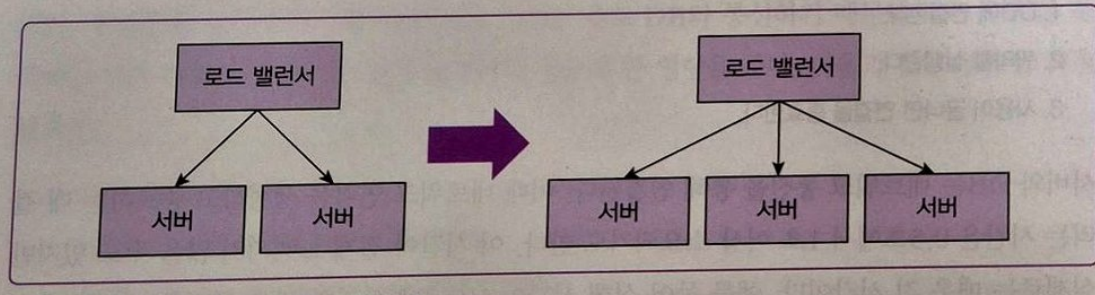


그림 2.9 수평 확장으로 처리량을 증가시킨다.

알아두기 **로드 밸런서**

서버가 두 대 이상이면 로드 밸런서load balancer가 필요하다. 로드 밸런서는 사용자 트래픽을 각 서버에 골고루 분배해서 한 서버에 사용자 트래픽이 몰리지 않도록 한다. 이를 통해 전체 서버 자원을 효율적으로 활용할 수 있다.

로드 밸런서가 트래픽을 알맞게 분산시키기 위해 사용하는 방식은 크게 정적인 방식과 동적인 방식으로 나뉜다.

정적인 방식의 대표적인 예로는 라운드 로빈과 IP 해시 방식이 있다. 라운드 로빈 방식은 클라이언트의 요청을 각 서버에 순차적으로 분배하는 방식이다. IP 해시 방식은 클라이언트의 IP 주소를 해시한 값을 기반으로 요청을 전달할 서버를 결정한다. IP 해시값은 동일하기 때문에 동일한 클라이언트는 항상 같은 서버로 연결되도록 해준다.

동적인 방식은 서버의 현재 상태에 따라 트래픽을 분산하는 방식으로 트래픽이 적은 서버에 요청을 보내는 형태로 동작한다. 예를 들어 연결 수가 더 적은 서버에 요청을 보내거나, 응답 시간이 더 짧은 서버에 요청을 보내는 식이다.

TPS를 높이기 위해 무턱대고 서버를 추가해서는 안 된다. 실제 병목 지점이 어디인지 파악하는 게 중요하다. DB에서 성능 문제가 발생하고 있는데 서버를 추가로 투입하면 불에 기름을 붓는 격이다. DB에 문제가 있는 상황에서 DB를 사용하는 서버를 더 늘리면 DB에 가해지는 부하가 더 커지고 성능 문제는 더 악화된다. 외부 API의 성능이 문제인 경우도 마찬가지다. 외부 API의 성능이 개선되지 않는 한 서버를 추가한다고 해도 TPS는 향상되지 않는다. DB나 외부 API에 성능 문제가 발생하지 않는 범위 내에서만 수평 확장을 해야 효과가 있다.

DB 커넥션 풀

DB를 사용하려면 다음과 같이 3단계를 거친다.

1. DB에 연결한다.
2. 쿼리를 실행한다.
3. 사용이 끝나면 연결을 종료한다.

서버와 DB는 네트워크 통신을 통해 연결된다. 이때 네트워크 연결을 생성하고 종료하는 데 걸리는 시간은 0.5초에서 1초 이상 소요되기도 한다. 이 시간이 길게 느껴지지 않을 수도 있지만 실제로는 매우 긴 시간이다. 예를 들어 실행 시간이 10ms에 불과한 짧은 쿼리를 실행하기 위해 연결과 종료에 50ms(0.05초)가 걸린다고 하자. 단순히 계산해도 전체 처리 시간인 60ms 중 80% 이상이 DB의 연결 및 종료에 쓰이게 된다.

네트워크에서 DB를 연결하고 종료하는 시간은 전체 응답 시간에 영향을 준다. 응답 시간이 길어지면 전체 처리량은 떨어진다. 트래픽이 증가하면 이러한 현상은 더 두드러진다. 매 요청마다 DB를 연결하고 종료하면 트래픽이 증가할 때 급격하게 처리량이 떨어지기도 한다.

이런 문제를 피하기 위해 DB 커넥션 풀을 사용한다. DB 커넥션 풀은 DB에 연결된 커넥션을 미리 생성해서 보관한다. 애플리케이션은 DB 작업이 필요할 때 풀에서 커넥션을 가져와 사용하고, 작업이 끝나면 다시 풀에 반환한다. 커넥션 풀을 사용하면 이미 연결된 커넥션을 재사용하기 때문에 응답 시간이 줄어드는 장점이 있다.

많이 사용하는 프레임워크나 언어도 DB 커넥션 풀을 지원한다. 서버 개발에서 DB 커넥션 풀을 사용하는 것은 필수라는 얘기다. 스프링 부트는 HikariCP를 커넥션 풀로 사용하며 Go 언어는 자체적으로 DB 커넥션 풀을 지원할 정도다.

커넥션 풀은 다양한 설정을 제공한다. 그중 중요한 설정은 다음과 같다.

- 커넥션 풀 크기(또는 최소 크기, 최대 크기)
- 풀에 커넥션이 없을 때 커넥션을 구할 때까지 대기할 시간
- 커넥션의 유지 시간(최대 유휴 시간, 최대 유지 시간)

각 설정이 성능과 품질에 어떤 영향을 주는지 차례대로 살펴보자.

커넥션 풀 크기

커넥션 풀 크기는 커넥션 풀에 미리 생성해둘 커넥션 개수를 지정하는 설정이다. 커넥션 풀 크기는 커넥션 풀 설정에서 가장 중요하다. 서버는 주로 DB와 통신하기 때문에, DB 연결을 관리하는 DB 커넥션 풀 크기를 잘못 설정하면 성능에 큰 영향을 준다. 다음과 같은 상황을 가정해보자.

- 커넥션 풀 크기는 5다.
- 한 요청에서 쿼리를 실행하는 데 1초 걸린다.
- 계산을 쉽게 하기 위해 데이터 전송 시간은 무시한다.

[그림 2.10]처럼 서버에 6개의 요청이 동시에 들어왔을 때 이 중 5개 요청은 풀에서 커넥션을 가져올 수 있다. 반면 나머지 1개 요청은 사용할 수 있는 커넥션이 없으므로 다른 요청이 커넥션 사용을 끝내고 풀에 반환할 때까지 기다려야 한다.

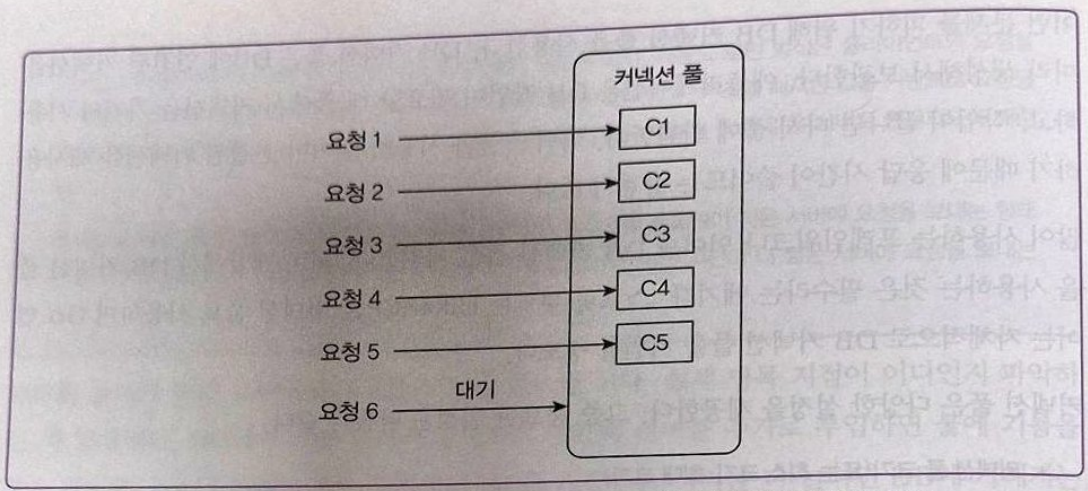


그림 2.10 풀의 모든 커넥션이 사용 중이면 다른 요청은 풀에 유휴 커넥션이 생길 때까지 대기하게 된다.

풀에서 커넥션을 얻기 위해 대기하는 시간을 줄이려면 전체 응답 시간과 TPS를 고려하여 커넥션 풀 크기를 지정해야 한다. 커넥션 풀 크기가 5이고 한 요청에서 쿼리를 실행하는 데 0.1초가 걸린다고 가정해보자. 이때 1초에 처리할 수 있는 요청 수는 50이 된다(1초/0.1초*5). 즉 동시에 50개의 요청이 들어오더라도 모두 1초 안에 처리가 끝난다. 물론 어떤 요청은 바로 커넥션을 얻어 실행되고, 어떤 요청은 커넥션을 구하기 위해 0.9초를 기다릴 수도 있지만 모두 1초 안에 처리할 수 있다.

반면 커넥션 풀 크기가 5이고 한 요청의 쿼리 실행 시간이 0.5초라면 1초에 처리할 수 있는 요청 수는 10으로 줄어든다(1초/0.5초*5). 이 상태에서 동시에 50개의 요청이 들어오면 모든 요청을 처리하는 데 5초가 걸리게 된다. 어떤 요청은 0.5초 만에 실행이 끝나지만, 다른 요청은 커넥션을 구하기 위해 4.5초를 기다렸다가 쿼리를 실행한다. 응답 시간 기준으로 5초는 꽤나 긴 시간이다. 이때는 응답 시간을 줄이기 위해 커넥션 풀 크기를 50으로 늘려 모든 요청을 0.5초 이내에 처리할 수 있다.

커넥션 풀의 구현 방식에 따라 다르지만 일반적인 커넥션 풀은 최소 크기와 최대 크기를 설정할 수 있다. 예를 들어 최소 크기를 10으로, 최대 크기를 20으로 지정했다고 하자. 동시에 들어오는 요청이 10개 이하면 커넥션 풀은 10개의 커넥션을 풀에 유지한다. 그러다 어느 시점에 동시에 들어오는 요청이 10개를 초과하면 커넥션 풀은 커넥션 수를 늘려서 요청을 처리한다. 동시 요청 수가 증가하면 커넥션 풀은 최대 크기인 20개까지 커넥션을 늘린다. 이후 동시 요청이 20개 이하로 줄면 커넥션 풀의 커넥션도 점차 줄어든다.

일반적으로 트래픽은 증가했다가 감소하는 패턴을 보인다. 은행 서비스는 낮 시간대에 트래픽이 높고, 게임 서비스는 저녁 시간대에 트래픽이 높은 경향이 있다. 이러한 특성에 맞게 커넥션 풀 크기도 트래픽이 적은 시간대는 최소 크기로 유지하고 트래픽이 높은 시간대에는 최대 크기로 확장해서 커넥션 개수를 필요한 만큼만 유지할 수 있다.

> Memo
>
> 트래픽이 순간적으로 급증하는 패턴을 보인다면 커넥션 풀의 최소 크기를 최대 크기에 맞추는 것이 좋다. 트래픽이 점진적으로 증가할 때는 DB 연결 시간이 성능에 큰 영향을 주지 않지만 트래픽이 급증할 경우 DB 연결 시간도 성능 저하의 주요 원인이 될 수 있기 때문이다.

커넥션 풀 크기를 늘리면 처리량을 높일 수 있다. 그러나 커넥션 풀 크기를 무턱대고 늘리면 안 된다. DB 상태를 보고 늘려야 한다. DB 서버의 CPU 사용률이 80%에 육박하는 상황에서 커넥션 풀 크기를 늘리면 DB에 가해지는 부하가 더 커져 쿼리 실행 시간이 급격히 증가할 수 있다. 이러한 상태에서는 커넥션 풀 크기를 늘리기보다는 오히려 커넥션 풀 크기를 유지하거나 줄여서 DB 서버가 포화 상태에 이르지 않도록 해야 한다. 서버를 수평 확장하는 것도 커넥션 풀 크기를 늘리는 것과 동일하다. DB 서버의 상태를 면밀히 확인한 후에 수평 확장을 진행해야 한다.

커넥션 대기 시간

대부분의 커넥션 풀은 대기 시간을 설정할 수 있다. 대기 시간이란 풀에 사용할 수 있는 커넥션이 없을 때 커넥션을 얻기 위해 기다릴 수 있는 최대 시간을 의미한다. 지정된 대기 시간 안에 커넥션을 구하지 못하면 DB 연결 실패 에러가 발생한다.

커넥션을 얻기 위해 대기하는 시간만큼 응답 시간도 길어진다. 참고로 HikariCP의 기본 대기 시간은 30초로 설정되어 있다. 이 말은 최악의 경우 응답 시간이 30초를 넘길 수도 있다는 것을 의미한다. 따라서 응답 시간이 중요한 서비스는 커넥션 대기 시간을 가능한 한 짧게 설정해야 한다. 트래픽의 양이나 서비스의 특성에 따라 차이는 있지만 보통의 경우라면 0.5초에서 3초 이내로 지정하자.

대기 시간을 짧게 설정하면 커넥션 풀이 모두 사용 중일 때 빠르게 '일시적 오류'와 같은 에러 응답을 사용자에게 보여줄 수 있다. 에러를 응답하는 게 부정적으로 보일 수도 있다. 하지만 대

기 시간 때문에 긴 시간 동안 무응답 상태로 유지되는 것보다는 빠르게 에러를 반환하는 것이 더 낫다. 커넥션을 얻지 못했을 때 빠르게 에러를 응답해야 서버의 부하가 증가하는 것도 방지할 수 있다.

예를 들어 커넥션 풀의 크기가 10이고 대기 시간이 30초라고 하자. 이때 동시에 30개의 요청이 발생했는데 순간적으로 DB 서버에 부하가 걸리면서 쿼리 실행 시간이 10초로 늘어났다. 이 시점에 각 요청은 다음 상태가 된다.

- 요청 10개는 풀에서 커넥션을 확보하여 쿼리 실행을 시작함
- 요청 20개는 풀에서 커넥션을 확보하지 못해 대기 상태로 진입함

대기하는 사람 중 절반이 기다리지 못하고 5초 만에 요청을 취소하고 다시 요청했다고 가정해 보자. 그러면 [그림 2.11]처럼 상태가 바뀌게 된다.

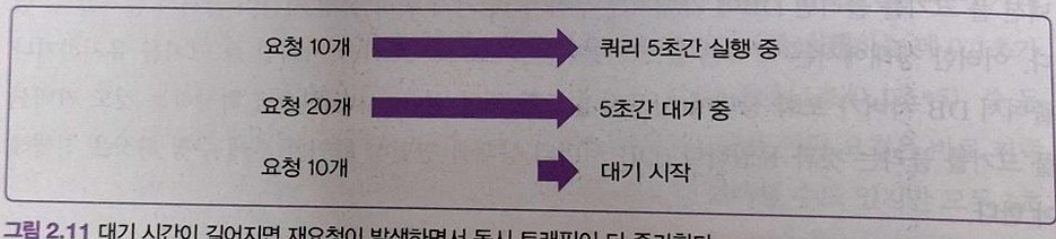


그림 2.11 대기 시간이 길어지면 재요청이 발생하면서 동시 트래픽이 더 증가한다.

풀에서 커넥션을 확보한 10개의 요청은 쿼리 실행 시간 10초 중 절반인 5초를 실행한 상태다. 풀에서 커넥션을 구하지 못한 나머지 20개의 요청은 여전히 대기 중이며 대기 시간 30초 중 5초가 흘러갔다. 그리고 이때 10명의 클라이언트가 5초 만에 요청을 취소하고 다시 요청을 보내면서 새로운 대기를 시작한다. 클라이언트가 요청을 취소하더라도 서버는 일정 시간 동안 하던 작업을 즉시 중단하지 않기 때문에 이 시점의 대기 중인 요청 수는 총 30개가 된다.

대기 중인 요청 수가 20개에서 30개로 증가하면서 서버가 동시에 처리해야 할 요청 수가 30개에서 40개로 늘었다. 이와 같이 몇 초 만에 요청을 취소하고 재요청이 반복되면 동시에 처리해야 할 요청 수는 계속 증가한다. 요청 수가 증가하면 그만큼 서버에 가해지는 부하도 커지게 된다.

커넥션 풀의 크기가 10개이고 대기 시간이 1초라면 어떻게 될까? 동일하게 동시에 30개의 요청이 들어오고 2초가 지난 시점에서의 상태는 다음과 같다.

- 풀에서 커넥션을 확보한 10개의 요청은 쿼리를 2초간 실행하고 있음
- 커넥션을 확보하지 못한 20개의 요청은 오류 응답을 받음

이 시점에 서버가 처리 중인 동시 요청 수는 10개이다. 새로운 요청이 들어오더라도 커넥션을 구하지 못하면 1초 뒤에 바로 에러를 응답한다. 이처럼 대기 시간을 짧게 설정하면 서버 부하를 일정 수준으로 유지할 수 있으며 서버를 안정적으로 운영하는 데 도움이 된다.

최대 유휴 시간, 유효성 검사, 최대 유지 시간

직원들이 사용하는 내부 업무 시스템을 떠올려보자. 근무 시간대에는 지속적으로 서버에 요청이 들어오지만 새벽 시간대에는 요청이 거의 없을 것이다. 요청이 없는 시간대에는 풀에 있는 커넥션도 사용되지 않는다. 이때 주의할 점이 있다. 커넥션이 사용되지 않는 시간이 길어지면 연결이 끊길 수 있다.

MySQL과 같은 DB는 클라이언트와 일정 시간 동안 상호작용이 없으면 자동으로 연결을 끊는 기능을 제공한다. 따라서 풀에 있는 커넥션이 일정 시간 이상 사용되지 않으면 DB와의 연결이 끊어질 수 있다. 예를 들어 DB가 1시간 동안 상호작용이 없는 클라이언트의 연결을 종료하도록 설정되어 있다고 가정해보자. 만약 새벽 시간대에 1시간 이상 사용자가 없으면 커넥션 풀에 있는 모든 커넥션은 DB와의 연결이 끊기게 된다.

이때 DB와의 연결이 끊긴 커넥션을 사용하면 에러가 발생한다. 이러한 연결 끊김으로 인해 발생하는 에러를 방지하기 위해 커넥션 풀은 다음 2가지 기능을 제공한다.

- 최대 유휴 시간 지정
- 유효성 검사 지원

최대 유휴 시간은 사용되지 않는 커넥션을 풀에 유지할 수 있는 최대 시간을 의미한다. 최대 유휴 시간을 30분으로 설정하면 30분 이상 사용되지 않은 커넥션은 종료되어 풀에서 제거된다. 이 시간을 DB에 설정된 비활성화 유지 시간보다 짧게 설정하면, DB가 연결을 끊기 전에 풀에서 커넥션을 제거할 수 있다.

유효성 검사는 커넥션이 정상적으로 사용할 수 있는 상태인지 여부를 확인하는 절차이다. 커넥션 풀의 구현 방식에 따라 커넥션을 풀에서 가져올 때 유효성을 검사하거나 주기적으로 검사할 수 있다. 이 과정을 통해 연결이 유효하지 않은 커넥션을 식별하고 풀에서 제거할 수 있다.

일부 커넥션 풀은 유효성 검사를 위해 실제 쿼리를 실행하기도 한다. 이때는 'SELECT 1 FROM dual'이나 'SELECT 1'과 같이 간단한 쿼리를 사용한다.

커넥션 풀이 제공하는 또 다른 설정은 최대 유지 시간이다. 이 값이 4시간으로 설정되어 있다면, 커넥션은 생성된 시점부터 최대 4시간까지만 유지된다. 4시간이 지나면 커넥션이 유효하더라도 커넥션을 닫고 풀에서 제거된다.

> Memo
> 최대 유휴 시간과 최대 유지 시간을 무한대로 설정하지 않는 것이 좋다. 커넥션 풀의 기본값을 확인한 뒤 이 두 설정의 기본값이 무제한으로 되어 있다면 DB 설정을 참고하여 알맞게 적절한 값으로 지정해야 한다.

서버 캐시

응답 시간을 줄이고 처리량을 높이기 위해 DB 서버를 수직 확장하거나 수평 확장할 수 있다. 하지만 DB 서버를 확장하려면 비용이 많이 든다. 또한 DB 서버를 수평 확장하더라도 처리량은 늘릴 수 있지만 실행 시간이 획기적으로 줄어들지는 않는다.

DB 서버를 확장하지 않고도 응답 시간과 처리량을 개선하고 싶다면 캐시cache 사용을 고려할 수 있다. 캐시는 일종의 (키, 값) 쌍을 저장하는 Map과 같은 형태의 데이터 저장소다. 캐시에 데이터를 저장해두면 동일한 데이터를 요청할 때 DB가 아닌 캐시에서 데이터를 읽어와 응답할 수 있다. 데이터 특성에 따라 차이는 있지만 일반적으로 캐시에서 데이터를 읽는 속도가 DB보다 빠르기 때문에 자주 조회되는 데이터를 캐시에 보관하면 응답 시간을 줄일 수 있다.

캐시의 전형적인 동작 방식은 [그림 2.12]와 같다.

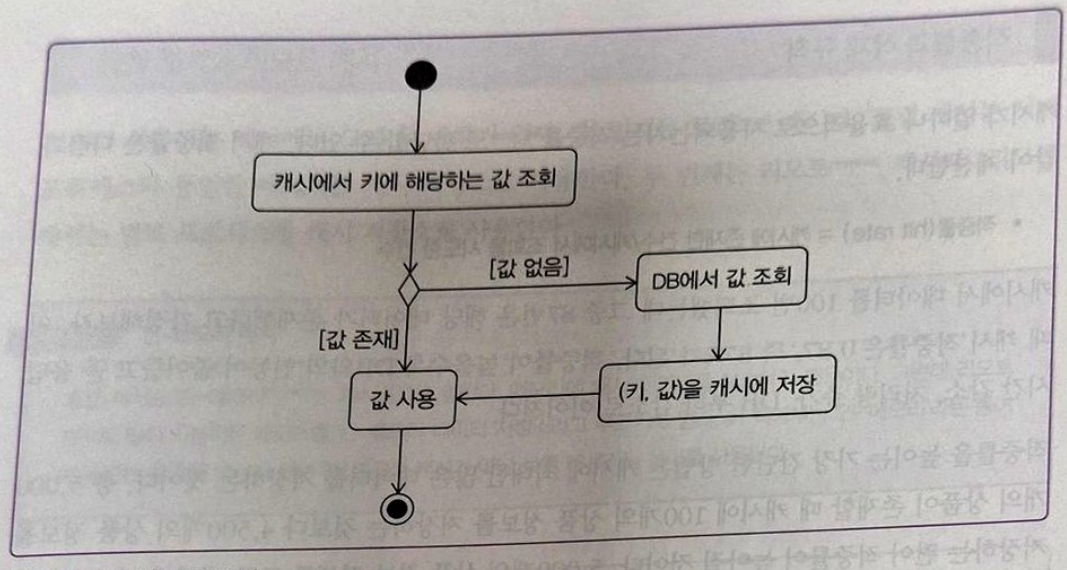


그림 2.12 캐시의 동작 방식

프로그램은 먼저 캐시에서 키에 해당하는 값을 조회한다. 값이 존재하면 바로 사용한다. 존재하지 않으면 DB 쿼리를 실행해 값을 조회한 후, 해당 값을 캐시에 저장하고 사용한다. 이후 동일한 요청이 들어오면 캐시에 저장된 값을 바로 사용한다. 캐시에서 데이터를 읽어오면 DB와의 연동 시간이 줄어 응답 시간이 단축된다.

DB뿐만 아니라 복잡한 계산 결과나 외부 API 연동 결과도 캐시에 보관하여 응답 시간을 줄이는 데 활용할 수 있다.

알아두기 **키 선택**

캐시는 (키, 값) 형태로 데이터를 저장하므로 저장하는 대상에 따라 적절한 캐시 키를 선택해야 한다. 예를 들어 게시글 상세 정보는 "articles:번호" 형태의 캐시 키를 사용하고 최신 인기 글은 "articles:hot10"을 키로 사용하는 식이다. 또한 캐시를 사용할 때는 캐시 키가 겹치지 않도록 주의해야 한다.

적중률과 삭제 규칙

캐시가 얼마나 효율적으로 사용되는지는 적중률hit rate로 판단할 수 있다. 캐시 적중률은 다음과 같이 계산한다.

- 적중률(hit rate) = 캐시에 존재한 건수/캐시에서 조회를 시도한 건수

캐시에서 데이터를 100번 조회했는데 그중 87번은 해당 데이터가 존재했다고 가정해보자. 이때 캐시 적중률은 0.87, 즉 87%가 된다. 적중률이 높을수록 DB와의 연동이 줄어들고 곧 응답 시간 감소, 처리량 증가, DB 부하 감소로 이어진다.

적중률을 높이는 가장 간단한 방법은 캐시에 최대한 많은 데이터를 저장하는 것이다. 총 5,000개의 상품이 존재할 때 캐시에 100개의 상품 정보를 저장하는 것보다 4,500개의 상품 정보를 저장하는 편이 적중률이 높아질 것이다. 5,000개의 상품 정보 전체를 모두 캐시에 저장하면 이론적으로 캐시 적중률은 100%가 된다.

하지만 캐시에 모든 데이터를 무작정 저장할 수는 없다. 캐시는 메모리 자원을 사용하기 때문이다. 사용할 수 있는 메모리 용량은 한계가 있기에 캐시에 저장할 수 있는 데이터 개수나 크기도 제한된다. 예를 들어 캐시에 최대 1,000개 데이터만 보관할 수 있다는 식이다.

캐시에 보관할 수 있는 데이터에 제한이 있으므로, 캐시가 가득 차 있는 상태에서 새로운 데이터를 캐시에 저장하면 기존에 있던 데이터 중 하나를 제거해야 한다. 삭제할 대상을 선택할 때 주로 사용하는 규칙은 다음과 같다.

- LRU(Least Recently Used): 가장 오래전에 사용된 데이터를 제거한다.
- LFU(Least Frequently Used): 가장 적게 사용된 데이터를 제거한다.
- FIFO(First In First Out): 먼저 추가된 데이터를 먼저 삭제한다.

많은 서비스에서는 오래된 데이터보다 최신 데이터를 더 자주 조회하는 경향이 있다. 최신 기사나 최신 영상의 조회수는 높지만 반대로 시간이 지난 데이터는 조회 수가 낮다. 즉, 오래된 데이터는 캐시에 있어도 사용되지 않을 가능성이 크다. 따라서 캐시가 가득 차 있지 않더라도 오래된 데이터는 미리 삭제하는 것이 좋다. 이렇게 하면 불필요한 메모리 사용을 줄일 수 있기 때문이다. 이를 위해 캐시에는 유효 시간(만료 시간)을 설정하는 방식도 함께 사용한다. 일정 시간이 지나면 캐시에서 해당 데이터를 자동으로 제거하여 메모리를 효율적으로 관리할 수 있다.

로컬 캐시와 리모트 캐시

서버가 사용하는 캐시에는 크게 두 종류가 있다. 첫 번째는 로컬local 캐시다. 로컬 캐시는 서버 프로세스와 동일한 메모리를 캐시 저장소로 사용한다. 두 번째는 리모트remote 캐시다. 리모트 캐시는 별도 프로세스를 캐시 저장소로 사용한다.

> **알아두기** 인-메모리 캐시
>
> 로컬 캐시는 인-메모리in-memory 캐시라고도 불린다. 메모리에 캐시 데이터를 보관하기 때문이다. 그런데 리모트 캐시로 많이 사용하는 레디스를 인-메모리 데이터 저장소라고 부르기도 한다. 이 책에서는 '인-메모리'라는 용어에서 오는 혼동을 줄이기 위해 인-메모리 캐시가 아닌 로컬 캐시라는 용어를 사용한다.

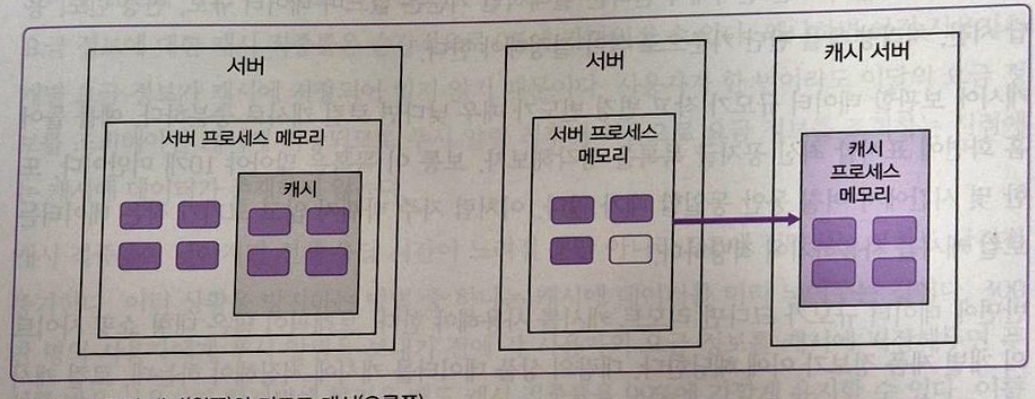


그림 2.13 로컬 캐시(왼쪽)와 리모트 캐시(오른쪽)

로컬 캐시 구현 기술로는 Caffeine(자바), go-cache(Go), node-cache(Node.js) 등이 있다. 로컬 캐시의 장점은 속도에 있다. 서버 프로세스와 캐시가 동일한 메모리 공간을 사용하므로 캐시 데이터에 빠르게 접근할 수 있다. 별도의 외부 연동이 필요하지 않아 구조를 단순하게 유지할 수 있다는 장점도 있다.

로컬 캐시의 단점은 캐시에 저장할 수 있는 데이터 크기에 제한이 있다는 점이다. 서버 프로세스가 사용할 수 있는 메모리양에 물리적인 한계가 있기 때문이다. 또한 서버 프로세스를 재시작하면 메모리에 존재하던 캐시 데이터가 모두 삭제되어 일시적으로 캐시 효율(적중률)이 순간적으로 떨어진다는 단점도 있다.

리모트 캐시는 로컬 캐시와 정반대의 장단점을 가진다. 가장 큰 장점은 캐시 크기를 유연하게 확장할 수 있다는 것이다. 대표적인 리모트 캐시 기술인 레디스Redis는 여러 대의 레디스 서버를 이용해서 수평 확장할 수 있는 기능을 제공한다. 이를 통해 캐시에 보관할 수 있는 데이터 규모를 늘릴 수 있다. 또한 서버 프로세스가 재시작되더라도 레디스에 저장된 캐시 데이터는 그대로 유지된다. 이런 장점 때문에 로컬 캐시와 대비해서 캐시 효율을 높일 수 있다.

리모트 캐시의 단점은 속도에 있다. 서버 프로세스는 캐시 프로세스와 데이터를 주고받기 위해 네트워크 통신을 해야 한다. 메모리에 저장된 데이터에 직접 접근하는 것과 비교하면 네트워크 통신은 상대적으로 느리다. 게다가 리모트 캐시를 운영하려면 별도의 서버 장비와 프로세스가 필요하기 때문에 시스템 구조가 복잡해진다.

로컬 캐시와 리모트 캐시는 각각 장단점이 뚜렷하기 때문에 상황이나 용도에 맞게 선택해야 한다. 어떤 방식을 무조건 선택해야 한다는 절대적인 기준은 없으며 데이터 규모, 변경 빈도, 응답 시간, 처리량 등을 판단 기준으로 삼아 결정해야 한다.

캐시에 보관할 데이터 규모가 작고 변경 빈도가 매우 낮다면 로컬 캐시로 충분하다. 예를 들어 홈 화면에 표시할 최신 공지글 목록을 생각해보자. 보통 이 목록은 많아야 10개 미만이다. 또한 몇 시간에서 며칠 동안 동일할 때가 많다. 이처럼 자주 바뀌지 않고 크기가 작은 데이터는 로컬 캐시를 사용하기에 적당하다.

반면에 데이터 규모가 크다면 리모트 캐시를 사용해야 한다. 트래픽이 많은 대형 쇼핑 사이트의 개별 제품 정보가 이에 해당한다. 대량의 상품 데이터를 캐시에 저장해야 하는데, 로컬 캐시로는 메모리 용량 등의 한계로 대응이 어렵다. 일부 데이터를 로컬 캐시에 저장하더라도 데이터가 수시로 변경되면 캐시 효율도 떨어진다.

배포 빈도가 높은 서비스라면(하루에도 몇 번씩 배포) 리모트 캐시 사용을 적극적으로 고려하자. 로컬 캐시를 사용할 경우 서버를 재시작될 때마다 캐시 데이터를 처음부터 다시 쌓아야 한다. 이로 인해 서버를 재시작한 직후 캐시 적중률이 떨어지고 응답 시간도 느려지게 된다. 리모트 캐시를 사용하면 서버를 재시작하더라도 캐시 데이터는 그대로 유지되므로 응답 시간을 일정 수준으로 유지할 수 있다.

캐시 사전 적재

트래픽이 순간적으로 급증하는 패턴을 보인다면 캐시에 데이터를 미리 저장하는 것도 고려할 필요가 있다. 다음은 캐시에 미리 데이터를 저장하면 큰 효과를 볼 수 있는 가상의 사례를 만들어 본 것이다.

- G 앱 사용자는 300만 명이다.
- G 앱 서비스는 사용자에게 매달 정해진 날에 이달의 요금 정보를 보여준다.
- 해당 일자가 되면 전체 회원을 대상으로 요금 안내 푸시 알림을 발송한다.
- 푸시를 받은 사용자 중 일부는 G 앱을 통해 이달의 요금 정보를 조회한다.

이 사례에서 푸시 알림을 받은 사용자는 G 앱을 실행해 즉시 요금 정보를 조회한다. 푸시를 받자마자 50%의 사용자가 바로 확인한다면, 단시간에 150만 명이 동시에 접속하게 된다. 이때 요금 정보에 대한 캐시 적중률은 순간적으로 0%에 가까워질 수 있다. 왜냐하면 아직 사용자의 개별 요금 정보가 캐시에 저장되어 있지 않기 때문이다. 사용자가 한 번이라도 이달의 요금 정보를 조회해야만 캐시에 저장되므로 푸시 알림 직후에 처음으로 요금 정보를 조회하는 시점에는 캐시에 데이터가 존재하지 않는다.

캐시 적중률이 낮아지면 전체 응답 시간이 느려질 뿐만 아니라 DB에 전달되는 부하도 급격히 증가한다. 이런 상황을 방지하는 방법 중 하나는 캐시에 데이터를 미리 넣어두는 것이다. 300만 명의 사용자에게 푸시 알림을 보내기 전에 각 사용자의 요금 정보를 캐시에 저장해두면 푸시를 받은 사용자가 한꺼번에 몰려올 때도 캐시 적중률을 99%에 가깝게 유지할 수 있다. 이를 통해 순간적으로 트래픽이 몰렸을 때도 응답 시간을 안정적으로 유지할 수 있으며, DB에 부하가 집중되는 현상도 효과적으로 방지할 수 있다.

캐시 무효화

캐시를 사용할 때 반드시 신경 써야 할 점은 유효하지 않은 데이터를 적절한 시점에 캐시에서 삭제하는 것이다. 캐시에 보관된 데이터의 원본이 바뀌면, 그에 맞춰 캐시에 보관된 데이터도 함께 변경하거나 삭제해야 한다. 원본이 변경됐는데 캐시에 저장된 데이터가 갱신되지 않으면 사용자는 오래된 잘못된 정보를 확인하게 되는 문제가 발생할 수 있다.

캐시에 저장된 데이터의 특성에 따라 캐시를 무효화하는 시점을 달리 설정해야 한다. 가격 정보, 게시글 내용처럼 민감한 데이터는 변경되는 즉시 캐시를 무효화해야 한다. 게시글 내용을 수정했는데도 캐시가 그대로 유지되면 사용자는 수정 전 게시글 내용을 보게 되어 혼란을 겪을 수 있다. 작성자는 수정이 제대로 반영되지 않았다고 생각해 오류 신고를 하거나 서비스에 대한 신뢰도를 낮게 평가할 수도 있다.

변경에 민감한 데이터는 로컬 캐시가 아닌 리모트 캐시에 보관해야 한다. 로컬 캐시는 자신의 데이터만 변경하지 다른 서버의 로컬 캐시는 변경하지 않기 때문이다. A 서버에 연결한 사용자는 변경된 가격 정보를 보지만 B 서버에 연결한 사용자는 B 서버의 로컬 캐시에 보관된 변경 전 가격 정보를 보게 되는 문제가 발생할 수 있어 서비스 신뢰성에 큰 영향을 줄 수 있다.

변경에 민감하지 않고 데이터 크기가 작다면 캐시의 유효 시간을 설정하여 주기적으로 갱신하는 방식을 사용해도 된다. 예를 들어 최근 인기 글 목록을 캐시에 저장한 경우, 최근 인기 글 목록이 바뀌고 몇 분 뒤에 캐시 데이터가 변경되더라도 서비스에 심각한 문제는 일어나지 않는다. 인기 글 목록을 저장하는 캐시의 유효 시간을 10분으로 지정하면 10분 주기로 최신 인기 게시글 목록을 갱신하는 효과를 얻을 수 있다. 갱신 시간을 줄이고 싶다면 유효 시간을 10분에서 1분으로 줄이기만 하면 된다.

가비지 컬렉터와 메모리 사용

자바, Go, 파이썬 등의 언어는 가비지 컬렉터를 사용한다. 가비지 컬렉터Garbage Collector를 사용하는 언어는 사용이 끝난 객체를 힙 메모리에서 바로 삭제하지 않고 정해진 규칙에 따라 사용하지 않는 메모리를 찾아서 반환한다. 예를 들어 힙 메모리 사용량이 일정 비율을 초과하면 가비지 컬렉터를 실행하거나, 일정 주기로 자동 실행된다.

가비지 컬렉터는 개발자가 메모리를 직접 관리해야 하는 부담을 줄여준다. 코드로 메모리를 관리할 수 없기 때문에 보안 이슈를 줄여주는 장점도 있다. 하지만 가비지 컬렉터는 응답 시간에 영향을 줄 수 있다. 예를 들어 자바에서는 가비지 컬렉터가 실행되는 동안 애플리케이션의 실행이 일시 중단되는데, 이것을 모든 실행이 멈춘다고 해서 'Stop-The-World'라고도 표현한다.

메모리를 많이 사용하고 생성된 객체가 많을수록 사용하지 않는 객체를 찾는 데 시간이 오래 걸린다. GC(Garbage Collection) 알고리즘과 메모리 사용 패턴에 따라 차이가 있지만 사용하는 메모리양과 객체 수가 많을수록 GC 실행 시간은 길어진다.

반대로 메모리 사용을 줄이면 GC 시간도 줄어들 가능성이 높아진다. 예를 들어 JVM에 할당된 최대 힙 크기를 4GB에서 2GB로 줄이면 GC가 탐지하고 제거해야 할 미사용 객체의 개수와 크기도 줄어든다. 검사해야 할 객체 수가 줄어드는 만큼 GC 수행 시간도 짧아진다. 물론 실제 애플리케이션이 4GB에 가까운 메모리가 있어야 하는 데도 2GB로 줄이면 메모리 부족으로 에러가 발생할 수 있으므로 실제 메모리 사용 패턴에 맞게 최대 힙 크기를 조정해야 한다.

한 번에 대량으로 객체를 생성하는 것도 주의해야 한다. 콘텐츠 조회 API가 한 번에 10만 개의 게시글을 반환할 수 있다고 하자. 게시글 하나가 0.5KB의 메모리를 사용한다면 10만 개의 게시글은 대략 50MB가 메모리를 차지하게 된다. 만약 동시에 100명 사용자가 조회 API를 호출하여 10만 개의 게시글을 조회한다면 약 4.9GB의 메모리가 필요해진다. 사용할 수 있는 최대 메모리가 4GB라면 메모리가 부족 상태가 발생한다. 메모리가 부족해서 GC를 실행하지만 메모리가 부족한 상태는 지속된다. 여전히 필요한 메모리는 4.9GB이기 때문이다.

대량으로 객체가 생성되는 것을 방지하려면 조회 범위를 제한해야 한다. 10년 치 거래 내역을 한 번에 조회할 수 있도록 하기보다는 최대 3개월 치만 거래 내역을 조회할 수 있게 하는 식으로 말이다. 마찬가지로 한 번에 조회할 수 있는 데이터의 개수도 트래픽 규모와 메모리 크기에 맞춰 제한해야 한다.

파일 다운로드와 같은 기능을 구현할 때는 스트림을 활용한다. 다음 자바 코드처럼 파일 데이터를 한꺼번에 메모리에 로딩한 후에 응답하는 방식은 피해야 한다. 파일 크기와 동시 사용자 수에 따라 메모리 사용량이 급증할 수 있기 때문이다. 이 코드는 30MB 크기의 파일을 100명이 동시에 다운로드하면 약 3GB의 메모리가 필요하게 된다.



```
byte[] bytes = Files.readAllBytes(Path.of("path")); // 파일을 한 번에 메모리에 로딩
out.write(bytes);
```

스트림을 활용하면 파일 처리 과정에서 필요한 메모리 크기를 줄일 수 있다. 다음은 스트림을 이용하도록 변환한 코드이다.

```
InputStream is = Files.newInputStream(Path.of("path"));
byte[] buffer = new byte[8192]; // 8KB 메모리
int read;
while ((read = is.read(buffer, 0, 8192)) >= 0) {
    out.write(buffer, 0, read);
}
// is.transferTo(out)와 동일 코드
```

이 코드는 파일을 한 번에 읽지 않고 8KB씩 끊어서 읽는다. 동시에 100명이 다운로드를 요청하더라도 필요한 메모리는 800KB에 불과하다. 파일 전체를 한 번에 메모리에 로딩하는 방식은 총 3GB에 가까운 메모리가 필요했는데, 이와 비교하면 스트림 방식은 0.001%도 안 되는 적은 메모리만을 사용하는 셈이다.

Column

대량 데이터를 한 번에 메모리에 올려서 서버가 중지된 사례

I사에서 일할 때 내부 사용자를 위한 관리 사이트에 접속할 수 없다는 연락을 받은 적이 있다. 관리 사이트용 서버는 2개였는데 그중 한 서버가 연결되지 않았다. 모니터링 시스템을 통해 확인한 결과 JVM에 할당한 최대 메모리 크기를 사용하고 있었으며 여러 차례 풀 GC가 실행되었지만 메모리는 계속해서 포화 상태에 머물렀다. 결국 메모리 부족 에러OOME, Out Of Memory Error도 발생했고, 문제 원인을 분석하기 위해 힙 덤프를 저장하고 서버를 재시작했다.

힙 덤프를 분석한 결과, 메모리에서 가장 높은 비중을 차지하고 있는 객체는 엑셀 생성과 관련된 객체였다. 누군가가 엑셀 다운로드 기능을 실행했는데 조회한 데이터가 수백만 건에 달했다. 엑셀 파일을 생성하는 과정에서 모든 데이터가 메모리에 올라갔다. 엑셀 생성 시간이 길어지자 사용자는 요청을 취소하고 다시 엑셀 다운로드를 시도했다. 동일한 데이터가 다시 메모리에 올라갔고, 이 과정에서 엑셀 생성을 위한 메모리 사용량이 급증하면서 메모리 부족 현상이 발생한 것이다.

이 문제를 해결하기 위해 2가지 조치를 취했다. 먼저 엑셀을 메모리에서 생성하지 않고 로컬 파일에 스트림 형태로 만들도록 변경했다. 둘째, 엑셀을 생성하기 위해 필요한 데이터를 조회하는 방식도 개선했다. 기존에는 DB에서 조회한 결과를 리스트로 한 번에 받아 처리했지만 스트림 형태로 받아 순차적으로 처리하는 방식으로 수정했다. 이러한 변경을 통해 엑셀을 생성하는 과정에서 필요한 메모리 사용량을 줄일 수 있었고 이후에는 동일한 문제가 발생하지 않았다.

Column

디스코드가 러스트로 간 이유

채팅 서비스로 유명한 디스코드는 성능 향상을 위해 기술을 적극적으로 수용하는 것으로도 잘 알려져 있다. 예를 들어 디스코드는 Go 언어로 구현된 서버를 러스트로 재구현한 적이 있다.[4] 이때 러스트로 전환한 주된 이유가 GC 때문이었다. GC로 인해 발생하는 응답 시간이 증가하거나 CPU 사용률이 급격히 튀는 문제가 발생했고, 이 문제를 해결하기 위해 디스코드 개발진은 GC가 없는 러스트를 선택했다. 그 결과 디스코드는 성공적으로 성능을 개선할 수 있었다.

이 사례를 듣고 '좋아, 이제 러스트로 다 개발해야지'라는 생각을 할 수도 있다. 하지만 러스트에 익숙한 개발자가 없다면 이런 생각은 일단 접어두자. 대량의 트래픽이나 높은 성능이 반드시 요구되는 상황이 아니라면 GC로 인한 성능 영향은 비교적 크지 않은 경우가 많으며, 시간이 지날수록 GC의 성능도 지속적으로 개선되고 있는 추세다. 일단 다른 성능 개선 방법이 있는지 검토해보자. 그럼에도 GC로 인한 영향을 줄일 수 없다면 그때 GC가 없는 언어를 도입하는 것을 고려하자.

응답 데이터 압축

응답 시간에는 데이터 전송 시간이 포함된다. 이 전송 시간은 2가지 요인에 영향을 받는다.

- 네트워크 속도
- 전송 데이터 크기

사용자의 네트워크 속도가 느리면 응답 시간이 길어진다. 하나의 무선 공유기에 너무 많은 사용자가 붙었을 때 웹사이트가 늦게 뜨는 것과 같다. 전송할 데이터의 크기가 커도 응답 시간이 길어진다. 10KB 크기의 파일을 다운로드받는 시간보다 1GB 크기의 파일을 다운로드하는 데 더 오랜 시간이 걸리는 것과 같다.

서버는 사용자의 네트워크 속도를 제어할 수 없지만 전송하는 데이터의 크기는 제어할 수 있다. 이때 사용할 수 있는 방법이 바로 응답 데이터를 압축해서 전송하는 것이다. 웹 서버가 전송하는 응답 데이터 중에서 HTML, CSS, JS, JSON과 같이 텍스트로 구성된 응답은 압축하면 데이터 전송량을 크게 줄일 수 있다. 실제로 텍스트 데이터를 gzip으로 압축하면 70% 이상 크기를 줄일 수 있으며 데이터 전송 크기가 줄어든 만큼 전송 시간도 빨라진다. 즉, 응답 시간이 짧아진다.

---

4 https://discord.com/blog/why-discord-is-switching-from-go-to-rust

응답 데이터를 압축해 데이터 전송량을 줄이는 것은 응답 시간뿐만 아니라 비용에도 영향을 준다. 클라우드 환경에서는 트래픽 자체가 비용으로 직결되기 때문이다. 따라서 응답 데이터 압축은 반드시 적극 검토해보자. 아파치나 Nginx와 같은 웹 서버는 압축 기능을 제공하고 있으므로 약간의 설정만 추가하면 즉시 효과를 볼 수 있다.

알아두기 **Accept-Encoding 요청 헤더와 Content-Encoding 응답 헤더**

웹 브라우저나 HTTP 클라이언트는 Accept-Encoding 헤더를 통해 서버에 처리할 수 있는 압축 알고리즘을 알린다. 예를 들어 gzip, deflate 알고리즘을 사용해서 압축을 풀 수 있다면 다음과 같이 Accept-Encoding 헤더를 전송한다.



```
Accept-Encoding: gzip, deflate
```

웹 서버는 Accept-Encoding 헤더에 명시된 알고리즘 중에서 자신이 지원하는 방식이 있을 경우, 해당 압축 알고리즘으로 응답 데이터를 압축해서 전송한다. 이때 사용된 압축 알고리즘은 Content-Encoding 응답 헤더를 통해 클라이언트에 전달된다.

응답 데이터를 압축할 때는 다음 사항을 고려하자.

- html, css, js, json과 같은 텍스트 형식의 응답은 압축률이 높아 효과적이다. 반면 jpeg 이미지나 zip 파일처럼 이미 압축한 데이터에는 다시 압축해도 효과가 없다. 따라서 모든 응답에 압축을 적용하지 말고 텍스트 형식의 데이터에 압축을 적용하자.
- 웹 서버에서 압축을 적용했더라도 방화벽이 이를 해제해 응답할 수 있다. 즉, 웹 서버에 압축 설정을 적용했음에도 실제 응답 데이터가 압축되지 않았다면 방화벽 설정도 확인해야 한다.

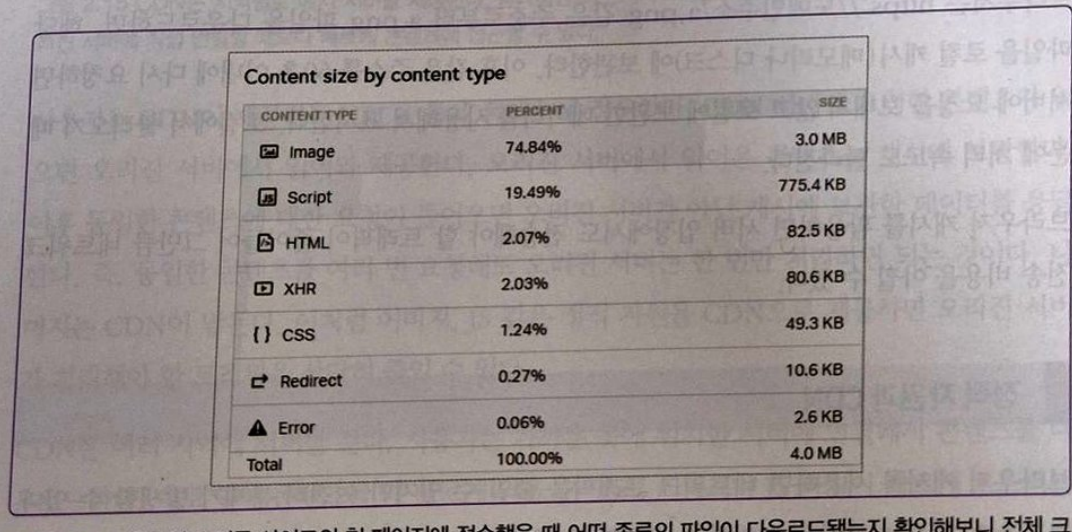
알아두기 **트래픽과 비용**

응답 데이터의 크기는 곧 비용으로 연결된다. AWS와 같은 클라우드 서비스는 트래픽 양에 비례해서 과금한다. 트래픽 규모가 작을 때는 금액 차이가 크지 않지만 트래픽 규모가 커질수록 데이터 크기를 줄이는 것만으로도 절감되는 비용이 커진다. IDC와 같은 온프레미스 환경을 사용할 때도 네트워크 대역폭에 따라 지불하는 비용이 달라진다. 트래픽이 증가하는 추세라면 응답 데이터의 크기를 줄일 수 있는지 확인해보자. 노력한 만큼 비용 절감 효과를 볼 수 있을 것이다.

정적 자원과 브라우저 캐시

온라인 쇼핑 사이트를 만들기 위해 서버를 개발했다고 가정해보자. 이 서버는 스프링 부트와 타임리프를 사용해서 HTML을 생성해 응답하고 JS와 CSS 파일도 함께 전달한다. JSON을 반환하는 API도 포함되어 있다. 또한 아파치 웹 서버를 통해 판매자가 업로드한 상품 이미지를 제공한다.

여기서 서버는 2가지 종류의 데이터를 응답한다. 첫 번째는 동적 자원이다. 동적 자원은 브라우저가 요청할 때마다 결과가 바뀌는 데이터로 제품 목록 HTML이나 제품 상세 JSON 응답이 여기에 해당한다. 두 번째는 정적 자원이다. 정적 자원은 같은 URL에 대해 같은 데이터를 응답하는 콘텐츠로 이미지, JS, CSS가 대표적이다. 정적 자원은 전체 트래픽에서 상당한 비중을 차지한다. 이미지가 많은 온라인 쇼핑몰 사이트의 첫 페이지는 정적 자원이 전체 데이터의 80%를 차지하기도 한다[5].



Content size by content type

CONTENT TYPE	PERCENT	SIZE
Image	74.84%	3.0 MB
Script	19.49%	775.4 KB
HTML	2.07%	82.5 KB
XHR	2.03%	80.6 KB
CSS	1.24%	49.3 KB
Redirect	0.27%	10.6 KB
Error	0.06%	2.6 KB
Total	100.00%	4.0 MB

그림 2.14 A 온라인 쇼핑몰 사이트의 첫 페이지에 접속했을 때 어떤 종류의 파일이 다운로드됐는지 확인해보니 전체 크기(4MB)에서 이미지와 스크립트(JS)가 약 94%(약 3.7MB)를 차지하고 있다. 이 데이터는 tools.pingdom.com을 이용해 측정한 결과다.

동일한 페이지에 들어갈 때마다 같은 이미지나 JS 파일을 매번 다운로드하면 서버 입장에서 좋을 게 없다. 트래픽은 비용과 연결된다. 더 많은 트래픽을 처리하려면 더 큰 비용을 내야 한다.

---

5 웹 브라우저의 개발자 도구를 사용하면 한 페이지를 불러오는 각 데이터 종류가 얼마나 많은 용량을 차지하는지 쉽게 확인할 수 있다.

한 사용자가 상품 상세 페이지를 10번 방문했을 때 동일한 이미지와 JS 파일을 10번 다운로드 받으면 그만큼 불필요한 트래픽이 발생하고 트래픽 비용도 증가할 수밖에 없다. 브라우저 입장에서도 비효율적이다. 같은 이미지와 JS를 매번 새로 받으면 그만큼 화면에 표시되는 속도도 느려지기 때문이다.

서버가 전송하는 트래픽을 줄이면서 브라우저가 더 빠르게 화면을 표시할 수 있는 방법이 있다. 바로 클라이언트 캐시를 활용하는 것이다. HTTP 프로토콜에서는 데이터를 응답할 때 Cache-Control이나 Expires 헤더를 이용해 클라이언트가 응답 데이터를 일정 시간 동안 저장해둘 수 있도록 설정할 수 있다. 예를 들어 https://도메인주소/a.png라는 이미지의 응답 헤더에 Cache-Control 헤더 값이 다음과 같이 지정되어 있다고 하자.



```
Cache-Control: max-age=60
```

브라우저는 https://도메인주소/a.png 같은 주소로부터 a.png 파일을 다운로드하면, 해당 파일을 로컬 캐시(메모리나 디스크)에 보관한다. 이후 같은 주소를 60초 이내에 다시 요청하면 서버에 요청을 보내지 않고 로컬에 보관한 데이터를 사용해서 표시한다. 로컬에서 불러오기 때문에 처리 속도도 빨라진다.

브라우저 캐시를 활용하면 서버 입장에서도 전송해야 할 트래픽이 줄어들어 그만큼 네트워크 전송 비용을 아낄 수 있다.

정적 자원과 CDN

브라우저 캐시를 사용하면 네트워크 트래픽을 줄일 수 있지만 여전히 문제가 발생할 수 있다. 브라우저 캐시는 브라우저 단위로 동작하기 때문에 동시에 많은 사용자가 접속하면 순간적으로 많은 양의 이미지, JS, CSS를 전송하게 된다. 이로 인해 빠르게 네트워크가 포화되어 응답 시간이 급격히 느려진다. 4차선 도로(대역폭)에 차가 1~2대씩 지나가면 막힘없이 잘 통과하다가 수십 대의 차가 몰리면 순식간에 도로가 막히는 것과 같다.

이런 문제를 해결하는 방법 중 하나는 CDN(Content Delivery Network, 콘텐츠 전송 네트워크)을 사용하는 것이다. 대표적인 CDN 서비스로는 Amazon CloudFront, Akamai, Cloudflare 등이 있다. 이름에서 알 수 있듯이 CDN은 콘텐츠를 제공하기 위한 별도의 네트워크를 의미한다. 이를 통해 클라이언트에 콘텐츠를 더 빠르고 효율적으로 전달할 수 있다. 일반적으로 CDN은 [그림 2.15]와 같은 구조를 갖는다.

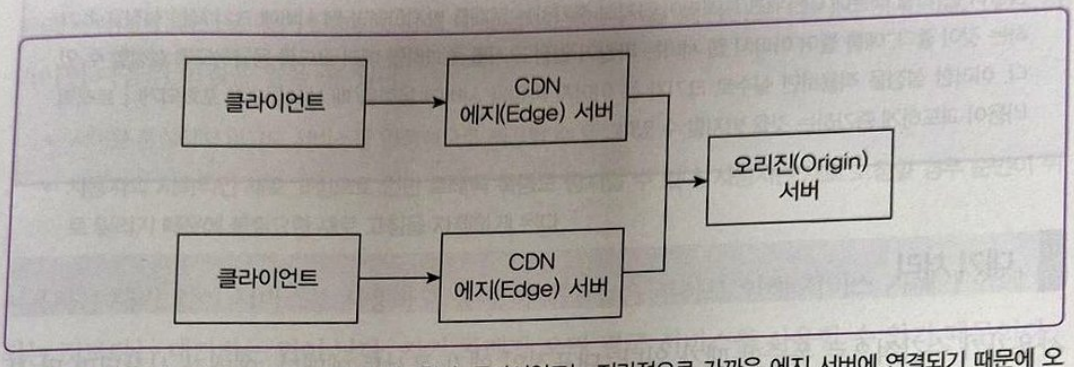


그림 2.15 CDN은 지역별로 에지 서버를 제공한다. 클라이언트는 지리적으로 가까운 에지 서버에 연결되기 때문에 오리진 서버에 직접 연결할 때보다 빠르게 콘텐츠에 접근할 수 있다.

사용자는 CDN이 제공하는 URL을 통해 콘텐츠에 접근한다. CDN 서버에 요청한 콘텐츠가 없으면 오리진 서버에서 읽어와 제공한다. 오리진 서버에서 읽어온 콘텐츠는 캐시에 보관한다. 이후 동일한 콘텐츠에 대한 요청이 들어오면 오리진 서버가 아닌 캐시에 보관한 데이터를 응답한다. 즉, 동일한 콘텐츠를 여러 번 요청해도 오리진 서버는 한 번만 처리하면 되는 것이다. 나머지는 CDN이 맡는다. 이처럼 이미지, JS 같은 정적 자원을 CDN으로 제공하면 오리진 서버가 처리해야 할 트래픽을 상당히 줄일 수 있다.

CDN은 여러 지역에 서버를 둔다. 사용자는 가까운 곳에 위치한 서버에 연결해서 콘텐츠를 다운로드한다. 이로 인해 콘텐츠를 더 빠르게 받을 수 있고, 이미지나 JS 파일이 빠르게 로드되는 만큼 브라우저도 더 빠르게 화면을 표시할 수 있다. 또한 CDN을 활용하면 오리진 서버에서 직접 콘텐츠를 제공하는 것보다 트래픽 비용도 적게 든다.

서비스가 성장하기 시작했다면 오리진 서버의 트래픽 감소, 콘텐츠의 빠른 제공, 트래픽 비용 절감이라는 장점이 있는 CDN 사용을 고려해보자.

알아두기 주의 사항

정적 파일을 관리할 때 주의할 점이 있다. 바로 크기다. 서비스를 운영하다 보면 실수로 용량이 큰 이미지 파일을 업로드할 때가 있다. 실제로 30MB나 되는 이미지 파일을 실수로 올려서 비용 청구가 올라간 사례도 있다. 계약 조건에 따라 특정 트래픽을 초과하지 못하게 막는 경우도 있다. 이때는 용량 초과로 서비스가 불능 상태에 빠지기도 한다. 따라서 이미지와 같은 파일을 업로드할 때는 파일 크기를 신중하게 관리해야 한다.

용량이 큰 파일 때문에 네트워크 트래픽이 급격히 증가하는 문제를 방지하려면 웹 서버에 크기 제한 설정을 추가하는 것이 좋다. 예를 들어 아파치 웹 서버는 파일이 일정 크기를 초과하면 에러 코드를 응답하도록 설정할 수 있다. 이러한 설정을 적용하면 실수로 크기가 큰 이미지 파일을 서버에 올렸을 때 네트워크가 포화되거나 트래픽 비용이 과도하게 증가하는 것을 방지할 수 있다.

대기 처리

사용자가 순간적으로 폭증할 때가 있다. 대표적인 예가 콘서트 예매다. 예매가 시작되면 몇 분 만에 매진이 되곤 한다. 많은 사용자가 표를 구매하기 위해 계속해서 클릭하고, 이로 인해 급격히 증가한 트래픽은 매진과 동시에 빠르게 감소한다. 1시간도 안 되는 짧은 시간 동안만 트래픽이 폭증하는 것이다.

이렇게 짧은 시간 동안 폭증하는 트래픽은 어떻게 처리해야 할까? 생각할 수 있는 방법 중 하나는 서버를 미리 증설하는 것이다. 클라우드를 사용하고 있다면 서버를 쉽게 증설할 수 있다. 하지만 서버만 증설한다고 끝나지 않는다. DB 성능도 문제가 된다. 트래픽은 DB에도 영향을 미치기 때문이다. DB는 쉽게 증설할 수 없기에 예상되는 트래픽에 맞춰 미리 증설해야 한다.

순간적으로 폭증하는 트래픽을 처리하기 위해 서버와 DB를 증설하는 것이 잘못된 방법은 아니지만 한 가지 문제가 있다. 바로 비용이다. 짧은 시간을 버티기 위해서 투입해야 하는 비용이 크기 때문이다. 클라우드에서 증설한 서버는 다시 줄일 수 있지만 DB는 그렇지 않다. 최대 트래픽에 맞춰 DB 성능을 높여 놓으면 다시 DB 성능을 줄이기가 쉽지 않다. 전체 서비스 시간 중 1%[6]도 되지 않는 시간을 위해 고정 비용(DB 비용)이 커지는 격이다.

---

6 30일 중에 예매 대란으로 트래픽이 폭증한 시간이 1시간이라고 해보자. 30일에서 1시간은 0.14%에 불과한 시간이다. DB와 같은 데이터 저장소는 상대적으로 비용이 많이 들기 때문에 0.14%를 위해 고정 비용을 높이는 것은 부담이 될 수 있다.

반대의 접근 방법도 있다. 처리할 수 있는 시스템의 처리량을 무작정 늘리기보다는 수용할 수 있는 수준의 트래픽만 받아들이고 나머지는 대기 처리하는 것이다. 대표적인 예로 사용자 대기 시스템일 들 수 있는데 은행 창구에서 고객을 응대하는 것과 비슷하다. 모든 창구에 고객이 있으면 나머지 고객은 기다린다. 한 창구의 고객이 일을 끝내면 대기 순번이 빠른 고객이 해당 창구로 가서 일을 처리한다.

트래픽이 순간적으로 증가할 때 동시에 수용할 사용자 수를 제한하고 나머지 사용자를 대기 처리하면 다음의 이점을 얻을 수 있다.

- 서버를 증설하지 않고도 서비스를 안정적으로 제공할 수 있다.
- 사용자의 지속적인 새로 고침으로 인한 트래픽 폭증도 방지할 수 있다. 사용자는 새로 고침할 경우 순번이 뒤로 밀리기 때문에 불필요한 새로 고침을 자제하게 된다.

사용자는 대기 없이 서비스를 사용하고 싶어 하지만 높은 부하로 인해 서비스 자체가 아예 안 되는 것보다는 대기하는 편이 낫다. 자기 차례가 오면 결국 서비스를 사용할 수 있기 때문이다.

> **Memo**
>
> 서버 개발자라면 누구나 한 번쯤 대규모 트래픽을 처리할 수 있는 아키텍처를 경험해보고 싶다는 마음이 생길 것이다. 하지만 개발자라면 비용도 생각해야 한다. 대규모 트래픽을 처리하기 위해 지불해야 할 비용은 결코 적지 않다. 이런 관점에서 대기 제어는 매우 합리적인 방식이라 할 수 있다. 게다가 대기 제어 기능을 직접 구현할 필요도 없다. 이미 여러 설루션이 존재하고 있어 빠르게 도입할 수 있다는 것도 큰 장점이다.